

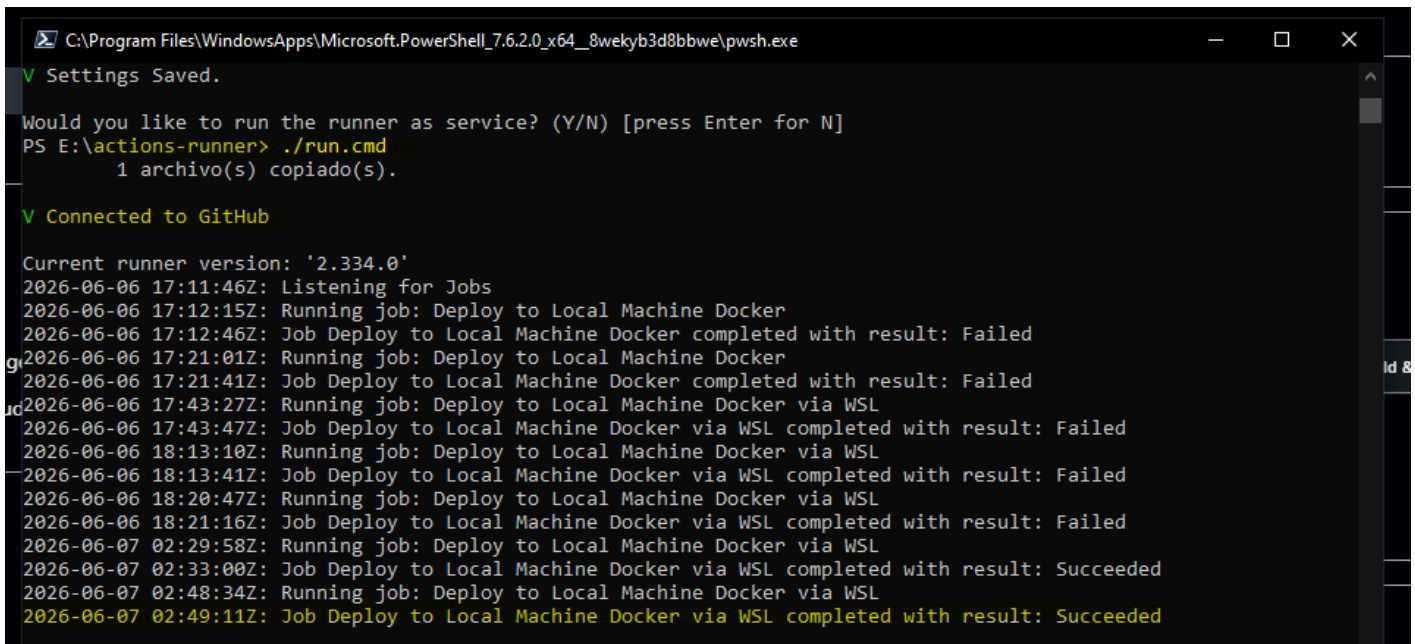
Documentación Técnica: Flujo de Despliegue Automatizado en Local

Documentación sobre despliegue automático en entorno local de un servicio empaquetado en una imagen de Docker, almacenada en GitHub Container Registry. Guiado por un Pipeline de GitHub Actions y ejecutado por un Self-hosted runner local, activándose automáticamente tras un Push o Pull Request sobre la rama main.

En la siguiente imagen podemos ver el servicio levantado localmente, siendo consumido por el navegador, con un mensaje de respuesta “Hello World!-TEST3”



Luego podemos ver la ejecución exitosa del Job de Despliegue en local, realizada por el Self-hosted runner local



Ahora vemos desde Docker Desktop el contenedor corriendo, el cual fue levantado por GitHub Action mediante el Self-hosted runner local.

Containers / ep1-backend-container

ep1-backend-container
ae791aff7431 gabto0011/ep1-users-service:latest
3080:3080

STATUS
Running (35 minutes ago)

Logs Inspect Bind mounts Exec Files Stats

View and search logs for all containers in one place with the new Logs view.

```
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [NestFactory] Starting Nest application...
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [InstanceLoader] ConfigHostModule dependencies initialized +17ms
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [InstanceLoader] AppModule dependencies initialized +0ms
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [InstanceLoader] ConfigModule dependencies initialized +0ms
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [InstanceLoader] UsersModule dependencies initialized +0ms
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RoutesResolver] AppController {/}: +5ms
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RouterExplorer] Mapped {/, GET} route +4ms
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RoutesResolver] UsersController {/users}: +0ms
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RouterExplorer] Mapped {/users, GET} route +1ms
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RouterExplorer] Mapped {/users/:id, GET} route +2ms
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RouterExplorer] Mapped {/users, POST} route +1ms
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RouterExplorer] Mapped {/users/:id, PATCH} route +2ms
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RouterExplorer] Mapped {/users/:id, DELETE} route +1ms
[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [NestApplication] Nest application successfully started +3ms
Server is running on port 3080
```

Podemos observar los logs al ejecutar el contenedor “ep1-backend-container”

2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [NestFactory] Starting Nest application...
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [InstanceLoader] ConfigHostModule dependencies initialized +17ms
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [InstanceLoader] AppModule dependencies initialized +0ms
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [InstanceLoader] ConfigModule dependencies initialized +0ms
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [InstanceLoader] UsersModule dependencies initialized +0ms
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RoutesResolver] AppController {/}: +5ms
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RouterExplorer] Mapped {/, GET} route +4ms
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RoutesResolver] UsersController {/users}: +0ms
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RouterExplorer] Mapped {/users, GET} route +1ms
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RouterExplorer] Mapped {/users/:id, GET} route +2ms
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RouterExplorer] Mapped {/users, POST} route +1ms
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RouterExplorer] Mapped {/users/:id, PATCH} route +2ms
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [RouterExplorer] Mapped {/users/:id, DELETE} route +1ms
2026-06-06 22:49:05	ep1-backend-container	[Nest] 1 - 06/07/2026, 2:49:05 AM LOG [NestApplication] Nest application successfully started +3ms
2026-06-06 22:49:05	ep1-backend-container	Server is running on port 3080

La siguiente imagen muestra la imagen “ep1-users-service” descargada de el GitHub Container Registry, la cual se utilizo para levantar el contenedor local “ep1-backend-container”.

Images Give feedback

Local My Hub

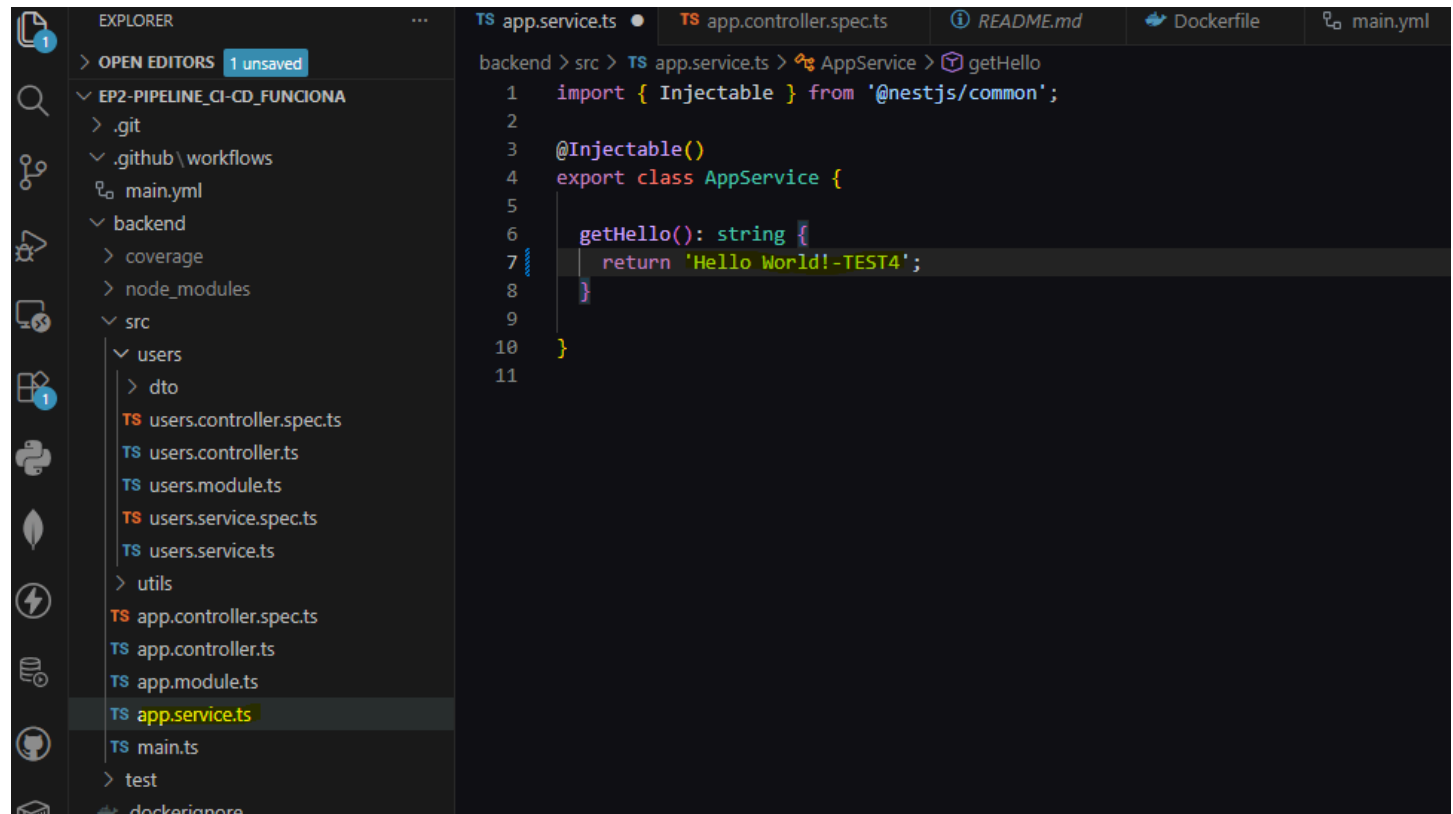
271.95 MB / 3.02 GB in use 5 Images Last refresh: 12 hours ago

Search

	Name	Tag	Image ID	Created	Size	Actions
☐	ep1-backend-test	latest	f96f8cb9e5bf	8 days ago	349.43 MB	
☐	ep1-database-test	latest	1d649a8a2f33	8 days ago	1.08 GB	
☐	ghcr.io/gabto0011/ep1-users-service	63630f75a21	fc1e8d835014	7 days ago	321.18 MB	
☐	ghcr.io/gabto0011/ep1-users-service	1.0.0	dd672f19ac5c	10 hours ago	321.18 MB	
☒	ghcr.io/gabto0011/ep1-users-service	latest	8d10db9ae29c	38 minutes ago	321.18 MB	

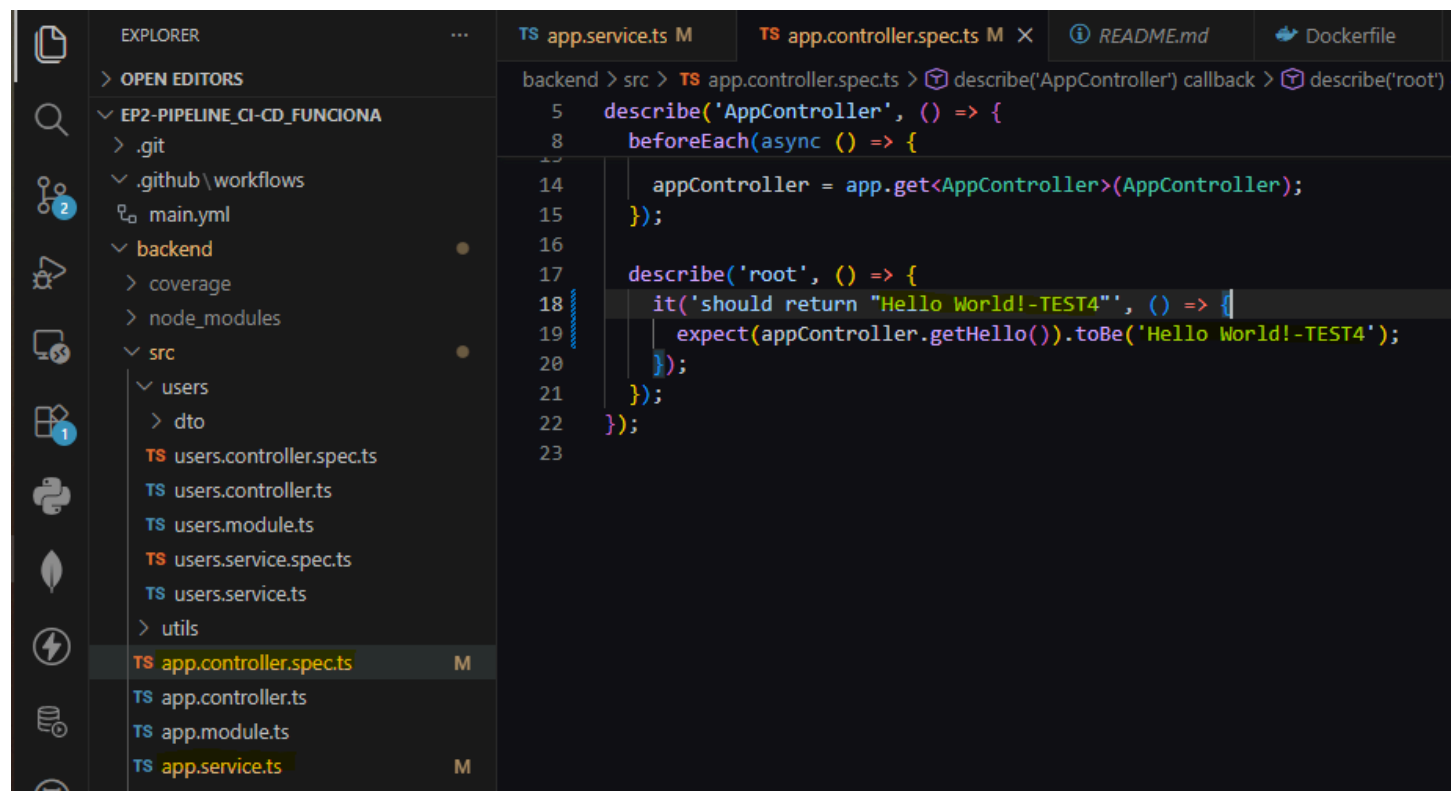
Ahora realizaremos un cambio en dos archivos del proyecto, para ver como se ve reflejado al activar el pipeline mediante un Push a la rama main.

Cambiamos el texto entregado por “Hello World!-TEST4”, además debemos cambiar los test respectivos, y así pasar el Job de pruebas en el pipeline.



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays the project structure, including folders like .git, .github, backend, and src. The file app.service.ts is selected and highlighted. The main editor area shows the code for app.service.ts, which includes an import for Injectable from '@nestjs/common', an @Injectable() decorator, and an export class AppService with a getHello() method that returns 'Hello World!-TEST4'.

```
backend > src > TS app.service.ts > AppService > getHello
1  import { Injectable } from '@nestjs/common';
2
3  @Injectable()
4  export class AppService {
5
6      getHello(): string {
7          return 'Hello World!-TEST4';
8      }
9
10 }
11
```



The screenshot shows the Visual Studio Code interface with the file app.controller.spec.ts selected. The main editor area displays the Jest test code for the AppController. The test includes a describe block for 'AppController', a beforeEach hook, and an it block that verifies the getHello() method returns 'Hello World!-TEST4'.

```
backend > src > TS app.controller.spec.ts > describe('AppController') callback > describe('root')
5  describe('AppController', () => {
8      beforeEach(async () => {
14
15          appController = app.get<AppController>(AppController);
16      });
17
18      describe('root', () => {
19          it('should return "Hello World!-TEST4"', () => {
20              expect(appController.getHello()).toBe('Hello World!-TEST4');
21          });
22      });
23  });
```

Ahora vemos la actualización de la rama mediante los comandos de git. Haciendo un push a la rama main, donde se activará el pipeline mediante el siguiente mensaje de commit “Despliegue Automatico Funcional - 2”

```

PS D:\Downloads\EP2-pipeline_CI-CD_funciona> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   backend/src/app.controller.spec.ts
        modified:   backend/src/app.service.ts

no changes added to commit (use "git add" and/or "git commit -a")
PS D:\Downloads\EP2-pipeline_CI-CD_funciona> git add .
warning: in the working copy of 'backend/src/app.controller.spec.ts', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'backend/src/app.service.ts', LF will be replaced by CRLF the next time Git touches it
PS D:\Downloads\EP2-pipeline_CI-CD_funciona> git commit -m "Despliegue Automatico Funcional - 2"
[main fdbc69d] Despliegue Automatico Funcional - 2
 2 files changed, 3 insertions(+), 3 deletions(-)
PS D:\Downloads\EP2-pipeline_CI-CD_funciona> git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 515 bytes | 515.00 KiB/s, done.
Total 6 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 5 local objects.
To https://github.com/GabTo0011/EP2-pipeline_CI-CD_funciona.git
 8f94339..fdbc69d  main -> main
PS D:\Downloads\EP2-pipeline_CI-CD_funciona>

```

Ahora podemos ver en el Repositorio en GitHub web, desde el apartado Actions, el nombre bajo el cual está corriendo el workflow “Despliegue Automatico Funcional - 2”

The screenshot shows the GitHub Actions interface for the repository `EP2-pipeline_CI-CD_funciona` by user `GabTo0011`. The **Actions** tab is active, displaying a list of workflows on the left sidebar, with `CI/CD + Security Pipeline` selected. The main content area shows the details of this workflow, including a **main.yml** file and a search bar for workflow runs. A banner prompts the user to **Help us improve GitHub Actions**. Below this, it indicates there are **25 workflow runs**. The most recent run, **Despliegue Automatico Funcional - 2**, is shown with a status of **Queued** and a commit hash of `fdbc69d` pushed by `GabTo0011`.

En la siguiente imagen podemos ver cómo se ejecuta el workflow, trabajo por trabajo.

The screenshot shows the GitHub Actions interface for a workflow named "Despliegue Automatico Funcional - 2 #25". The workflow is triggered by a push to the main branch. The status is "In progress". The summary shows it was triggered via push 1 minute ago, by user GabTo0011, with 2 artifacts. The workflow steps are: Snyk Security Scan (20s), Run Tests and Coverage (22s), SonarCloud Analysis (13s), Build & Push Docker Image, Deploy to Simulated Cloud (Compose), and Deploy to Local Machine Docker. The first three steps are completed, and the last three are in progress.

Luego podemos ver que el workflow se ejecutó con éxito cada trabajo desde el escaneo de seguridad hasta el despliegue en local.

The screenshot shows the same GitHub Actions workflow, now completed successfully. The status is "Success". The summary shows it was triggered via push 4 minutes ago, by user GabTo0011, with 2 artifacts. The workflow steps are: Snyk Security Scan (20s), Run Tests and Coverage (22s), SonarCloud Analysis (13s), Build & Push Docker Image (32s), Deploy to Simulated Cloud (Compose) (1m 38s), and Deploy to Local Machine Docker via WSL (3s). All steps are completed.

Desde el Self-hosted runner local podemos visualizar que el Job de Despliegue “Deploy to Local Machine Docker” se ejecutó completamente con éxito.

```
C:\Program Files\WindowsApps\Microsoft.PowerShell_7.6.2.0_x64_8wekyb3d8bbwe\pwsh.exe
V Settings Saved.

Would you like to run the runner as service? (Y/N) [press Enter for N]
PS E:\actions-runner> ./run.cmd
    1 archivo(s) copiado(s).

V Connected to GitHub

Current runner version: '2.334.0'
2026-06-06 17:11:46Z: Listening for Jobs
2026-06-06 17:12:15Z: Running job: Deploy to Local Machine Docker
2026-06-06 17:12:46Z: Job Deploy to Local Machine Docker completed with result: Failed
2026-06-06 17:21:01Z: Running job: Deploy to Local Machine Docker
2026-06-06 17:21:41Z: Job Deploy to Local Machine Docker completed with result: Failed
2026-06-06 17:43:27Z: Running job: Deploy to Local Machine Docker via WSL
2026-06-06 17:43:47Z: Job Deploy to Local Machine Docker via WSL completed with result: Failed
2026-06-06 18:13:10Z: Running job: Deploy to Local Machine Docker via WSL
2026-06-06 18:13:41Z: Job Deploy to Local Machine Docker via WSL completed with result: Failed
2026-06-06 18:20:47Z: Running job: Deploy to Local Machine Docker via WSL
2026-06-06 18:21:16Z: Job Deploy to Local Machine Docker via WSL completed with result: Failed
2026-06-07 02:29:58Z: Running job: Deploy to Local Machine Docker via WSL
2026-06-07 02:33:00Z: Job Deploy to Local Machine Docker via WSL completed with result: Succeeded
2026-06-07 02:48:34Z: Running job: Deploy to Local Machine Docker via WSL
2026-06-07 02:49:11Z: Job Deploy to Local Machine Docker via WSL completed with result: Succeeded
2026-06-07 03:35:37Z: Running job: Deploy to Local Machine Docker via WSL
2026-06-07 03:36:06Z: Job Deploy to Local Machine Docker via WSL completed with result: Succeeded

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, action
```

Ahora vemos que el Job bajo la nueva imagen “ep1-users-service” latest desde el GitHub Container Registry.

Images

LocalMy Hub

271.95 MB / 3.02 GB in use5 Images

Last refresh: 12 hours ago

Search

	Name	Tag	Image ID	Created	Size	Actions
	ep1-backend-test	latest	f96f8cb9e5bf	8 days ago	349.43 MB	
	ep1-database-test	latest	1d649a8a2f33	8 days ago	1.08 GB	
	ghcr.io/gabto0011/ep1-users-service	63630f75a21	fc1e8d835014	7 days ago	321.18 MB	
	ghcr.io/gabto0011/ep1-users-service	1.0.0	dd672f19ac5c	10 hours ago	321.18 MB	
	ghcr.io/gabto0011/ep1-users-service	latest	1de1f9adce80	6 minutes ago	321.18 MB	

Podemos ver en la siguiente imagen el nuevo contenedor levantado con la imagen descargada.

Containers

Container CPU usage0.00% / 400% (4 CPUs available)

Container memory usage22.47MB / 7.52GB

Show charts

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
	ep1-backend-conta	1862b8d6beab	ghcr.io/gabto0011/ep1-users-service:latest	3080:3080	0%	4 minutes ago	

Ahora podemos revisar los logs de funcionamiento, y ver que el servicio responde correctamente.

Containers / ep1-backend-container

ep1-backend-container

1862b8d6beab gabo0011/ep1-users-service:latest 3080:3080

STATUS Running (5 minutes ago)

Logs Inspect Bind mounts Exec Files Stats

View and search logs for all containers in one place with the new Logs view.

```
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [NestFactory] Starting Nest application...
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [InstanceLoader] ConfigHostModule dependencies initialized +19ms
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [InstanceLoader] AppModule dependencies initialized +1ms
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [InstanceLoader] ConfigModule dependencies initialized +0ms
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [InstanceLoader] UsersModule dependencies initialized +0ms
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RoutesResolver] AppController {}: +14ms
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RouterExplorer] Mapped {/, GET} route +3ms
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RoutesResolver] UsersController {/users}: +0ms
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RouterExplorer] Mapped {/users, GET} route +1ms
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RouterExplorer] Mapped {/users/:id, GET} route +2ms
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RouterExplorer] Mapped {/users, POST} route +1ms
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RouterExplorer] Mapped {/users/:id, PATCH} route +2ms
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RouterExplorer] Mapped {/users/:id, DELETE} route +3ms
[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [NestApplication] Nest application successfully started +5ms
Server is running on port 3080
```

Time	Container	Log
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [NestFactory] Starting Nest application...
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [InstanceLoader] ConfigHostModule dependencies initialized +19ms
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [InstanceLoader] AppModule dependencies initialized +1ms
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [InstanceLoader] ConfigModule dependencies initialized +0ms
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [InstanceLoader] UsersModule dependencies initialized +0ms
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RoutesResolver] AppController {}: +14ms
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RouterExplorer] Mapped {/, GET} route +3ms
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RoutesResolver] UsersController {/users}: +0ms
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RouterExplorer] Mapped {/users, GET} route +1ms
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RouterExplorer] Mapped {/users/:id, GET} route +2ms
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RouterExplorer] Mapped {/users, POST} route +1ms
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RouterExplorer] Mapped {/users/:id, PATCH} route +2ms
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [RouterExplorer] Mapped {/users/:id, DELETE} route +3ms
2026-06-06 23:36:00	ep1-backend-container	[Nest] 1 - 06/07/2026, 3:36:00 AM LOG [NestApplication] Nest application successfully started +5ms
2026-06-06 23:36:00	ep1-backend-container	Server is running on port 3080

113 lines

Finalmente podemos recargar la página con F5 o el botón del navegador y ver que el mensaje entregado al consumir el servicio cambio a “Hello World!-TEST4”

localhost:3080

Hello World!-TEST4